Conteúdo 8

Site: AVA - Acadêmico do ZL/IFRN
Curso: Técnicas de Análise de Dados
Livro: Conteúdo 8

Impresso por: Raul Vitor de Lima Barra
Data: sábado, 2 mai. 2026, 15:23

Índice

1. Introdução à Aprendizagem Supervisionada

2. Algoritmos - Regressão Logística e Árvores de Decisão

3. Pipeline de Dados e Avaliação de Desempenho

4. Métricas de Avaliação e Matriz de Confusão

5. Interpretabilidade, AutoML e Hiperparâmetros

6. Implementação Prática com Scikit-Learn

7. Automação com AutoML (PyCaret e AutoGluon)

8. Infográfico da Unidade 8

1. Introdução à Aprendizagem Supervisionada

Neste capítulo introdutório, damos início ao último tópico do curso de análise de dados, focado na Aprendizagem Supervisionada. É fundamental estabelecer, logo de início, que este é um campo vasto e complexo. O conteúdo abordado nesta disciplina serve como uma introdução a um tema que, por si só, preencheria disciplinas inteiras de pós-graduação, mestrado ou doutorado, com cargas horárias extensas de 80 horas ou mais. Existem inúmeros detalhes, nuances e uma grande variedade de algoritmos que não serão aprofundados exaustivamente neste momento. O objetivo principal aqui é fornecer uma compreensão introdutória sobre aprendizagem supervisionada, capacitando o aluno a gerar códigos que construam modelos preditivos, mas sem a pretensão de esgotar o assunto. A curiosidade e o estudo contínuo além deste material são essenciais para quem deseja dominar a área.

A definição central de aprendizagem supervisionada baseia-se na utilização de um histórico de dados de um fenômeno observado. Esse histórico é composto por um conjunto de variáveis de entrada — também chamadas de variáveis independentes — e uma variável de saída, conhecida como variável dependente ou variável alvo. O objetivo primordial dos algoritmos desta classe é produzir um modelo, uma hipótese ou uma função capaz de relacionar essas variáveis de entrada com a variável de saída. Diferentemente da aprendizagem não supervisionada (abordada em módulos anteriores), na supervisionada nós temos um rótulo, um objetivo claro que queremos prever.

Podemos ilustrar esse conceito com um exemplo prático no contexto educacional. Imagine uma base de dados onde as variáveis de entrada contêm informações sobre alunos, como sua caracterização social, dados demográficos e histórico educacional. A variável de saída, ou o nosso alvo, poderia ser o desempenho desse aluno na disciplina de português ou sua situação de matrícula (evadido ou persistente). Ao entrar um novo grupo de alunos na instituição, o objetivo é utilizar o modelo treinado com os dados históricos para prever o desempenho ou o comportamento desses novos estudantes.

A natureza da variável de saída determina o tipo de problema que estamos resolvendo. Quando a variável de saída é do tipo numérico — por exemplo, prever a temperatura daqui a três dias ou o valor do Real daqui a cinco dias — estamos lidando com um processo de **regressão**. Por outro lado, caso a variável de saída seja do tipo categórico — como classificar um aluno entre "evadido" ou "permanente", ou determinar se um cliente é "pagador" ou "devedor" — estamos diante de um processo de **classificação** de dados.

Visualmente, podemos imaginar uma base de dados onde certas colunas representam o nosso X (entradas) e uma coluna específica representa o nosso Y (saída). O algoritmo de aprendizagem supervisionada processa esse X para tentar mapear e prever o Y. O processo de treinamento envolve uma função de custo (loss function), que mede o erro do modelo. A partir da mensuração desse erro, o algoritmo atualiza seus parâmetros internos de forma iterativa. Ele repete esse processo várias e várias vezes, ajustando o modelo até que ele consiga, dada uma entrada, prever a saída com a menor margem de erro possível.

Existem diversos algoritmos de aprendizagem supervisionada, todos compartilhando esse mesmo propósito de mapeamento. No "Machine Learning clássico", os principais algoritmos incluem a Regressão Linear, a Regressão Logística, as Árvores de Decisão, as Florestas Randômicas (Random Forests), as SVMs (Support Vector Machines), o K-Vizinhos Mais Próximos (KNN) e o Naive Bayes (baseado na teoria bayesiana). Além destes, temos as Redes Neurais e técnicas de "Ensemble" (como o AdaBoost), que combinam vários classificadores para melhorar a predição.

Cada um desses algoritmos possui mecanismos específicos de funcionamento. Durante o treinamento, eles iteram sobre os dados de entrada. Inicialmente, os parâmetros podem ser aleatórios, gerando saídas ruins. No entanto, através da função de custo específica de cada algoritmo, o sistema verifica se acertou ou errou e atualiza os pesos ou regras. O framework *Scikit-Learn*, que utilizaremos, é uma ferramenta poderosa que já possui praticamente todos esses algoritmos e suas funções de custo implementados, facilitando imensamente o trabalho do cientista de dados.

A estrutura do nosso estudo neste tópico cobrirá a definição de aprendizagem, exemplos de algoritmos, a pipeline completa de dados, a divisão crucial entre conjuntos de treino, teste e validação, e os conceitos de viés (bias) e variância. Também discutiremos como avaliar esses modelos, a importância da interpretabilidade e explicabilidade, e faremos uma breve passagem sobre AutoML e otimização de hiperparâmetros. É uma jornada que visa transformar dados históricos em previsões futuras úteis para a tomada de decisão.

Unidade 8.1

Tecnologia em Sistemas para Internet

Assista no

2. Algoritmos - Regressão Logística e Árvores de Decisão

Dando continuidade ao entendimento da aprendizagem supervisionada, este capítulo aprofunda-se no funcionamento de dois algoritmos clássicos: a Regressão Logística e as Árvores de Decisão. O propósito fundamental permanece o mesmo: dado um conjunto de variáveis de entrada (independentes), queremos mapeá-las em uma variável de saída (dependente) para prever o futuro ou categorizar uma situação, auxiliando na tomada de decisões de negócio, como prever risco de inadimplência ou a temperatura futura.

A **Regressão Logística** é um algoritmo amplamente utilizado para problemas de classificação. Embora o nome contenha "regressão", seu uso principal é classificar dados em categorias. O funcionamento interno baseia-se em uma equação matemática que gera uma reta (ou hiperplano, em dimensões superiores). O objetivo do algoritmo durante o treinamento é encontrar os pesos (parâmetros) ideais para essa equação de tal forma que maximize a probabilidade de separação entre as classes.

Visualizando uma simulação, podemos observar uma reta se movendo e se ajustando conforme os pesos são atualizados. Esses pesos multiplicam as variáveis de entrada. Por exemplo, se tivermos "nota de português" e "nota de matemática" como entradas, cada uma será multiplicada por um peso específico, somadas a um valor de viés (bias), e o resultado passará por uma função que transforma esse valor em uma probabilidade entre 0 e 1. Se o resultado final for acima de um determinado limiar (threshold), comumente 0.5, o modelo classifica a instância como Classe 1 (por exemplo, aluno evadido). Se for abaixo, classifica como Classe 0 (aluno persistente). A "aprendizagem" aqui é justamente o ajuste automático desses valores numéricos (pesos) para minimizar o erro de classificação nos dados de treino.

Por outro lado, temos as **Árvores de Decisão**, que operam de maneira lógica e hierárquica, assemelhando-se a um fluxograma de regras "se-então" (if-else). O algoritmo analisa os atributos de entrada e tenta encontrar pontos de corte que melhor separam os dados. Por exemplo, o algoritmo pode determinar que, se a "nota de português" for menor que 0,45 e a "nota de matemática" for maior que 0,25, o aluno pertence à Classe 0. Caso contrário, verifica-se uma nova regra.

O treinamento de uma árvore de decisão envolve a criação dessas regras de divisão. O algoritmo busca, a cada passo, a divisão que torna os subconjuntos de dados resultantes mais "puros" ou homogêneos em relação à variável de saída. Diferente da regressão logística que ajusta pesos numa equação, a árvore ajusta os pontos de corte e a estrutura hierárquica das perguntas.

É crucial distinguir dois conceitos que surgem aqui: **Parâmetros** e **Hiperparâmetros**.

- **Parâmetros** são as configurações internas que o modelo aprendeu *após* o treinamento. No caso da regressão logística, são os pesos (ex: 2.53, 3.10) e o viés. No caso da árvore de decisão, são as regras de corte e a estrutura da árvore gerada.
- **Hiperparâmetros** são as configurações definidas *antes* do treinamento, que controlam como o algoritmo aprende. Na regressão logística, um exemplo é a taxa de aprendizagem. Na árvore de decisão, um hiperparâmetro clássico é a "profundidade máxima" da árvore. Se definirmos uma profundidade máxima de 2, a árvore será pequena e simples; se definirmos 6, ela será complexa e profunda. Esses hiperparâmetros influenciam diretamente o resultado final do modelo e podem ser ajustados para melhorar o desempenho.

A complexidade desses algoritmos é vasta. Para entender completamente como a função de perda opera matematicamente ou como exatamente um algoritmo decide o melhor ponto de corte numa árvore, seria necessário um estudo matemático aprofundado. No entanto, a mensagem central é que, independentemente da complexidade matemática interna, esses algoritmos servem como ferramentas para otimizar uma função que traduz entradas em saídas, permitindo a automação de previsões baseadas em dados históricos.

Unidade 8.2

Tecnologia em Sistemas para Internet

Assista no

3. Pipeline de Dados e Avaliação de Desempenho

Neste capítulo, expandimos a visão para a pipeline completa de dados em um projeto de Machine Learning, focando em etapas críticas como o pré-processamento, a divisão dos dados e o entendimento dos erros de viés e variância.

A construção de um modelo de aprendizagem supervisionada não se resume apenas ao treinamento do algoritmo. Existe um fluxo de trabalho (pipeline) que deve ser dominado pelo cientista de dados ou engenheiro de machine learning. O processo inicia-se com o **entendimento do problema** e do negócio, seguido pela **análise exploratória** dos dados (EDA), onde gráficos e estatísticas nos ajudam a compreender a natureza das informações. Em seguida, vem o **pré-processamento**, que inclui limpeza de dados, imputação de valores ausentes e balanceamento de classes. A **engenharia e seleção de atributos** também são etapas vitais para refinar o que será entregue ao modelo. Só então entramos na fase de **modelagem**, seguida pela **avaliação** e, por fim, a **interpretabilidade** e deploy.

Um conceito fundamental introduzido aqui é a **Segregação de Dados**. Nunca devemos treinar e testar um modelo com o mesmo conjunto de dados, pois isso seria equivalente a "roubar" ou decorar as respostas, gerando o que chamamos de vazamento de dados (data leakage). A prática correta é dividir os dados brutos (raw data) em dois conjuntos distintos: **Treino** e **Teste**.

- O conjunto de **Treino** é usado para o algoritmo aprender e ajustar seus parâmetros.
- O conjunto de **Teste** é guardado e usado apenas no final para verificar se o modelo generaliza bem para dados que nunca viu antes.

As proporções dessa divisão variam. Historicamente, usava-se 70% para treino e 30% para teste. Com o advento do Big Data e bases com terabytes de informação, é comum vermos divisões como 98% para treino e 2% para teste, pois mesmo 1% de uma base massiva já é suficiente para validar o modelo. Quando temos poucos dados, utilizamos técnicas como a **Validação Cruzada (Cross-Validation)**, onde dividimos os dados em, por exemplo, 5 partes (folds), treinando em 4 e testando em 1, rotacionando esse processo 5 vezes e tirando a média da performance.

Ao avaliar o desempenho, nos deparamos com o trade-off entre **Viés (Bias)** e **Variância**.

- **Underfitting (Alto Viés):** Acontece quando o modelo é simples demais para capturar os padrões complexos dos dados. Ele erra muito tanto no treino quanto no teste, ou seja, o modelo não aprendeu o suficiente. A solução geralmente envolve aumentar a complexidade do modelo (ex: mais neurônios numa rede neural, mais profundidade numa árvore).
- **Overfitting (Alta Variância):** Ocorre quando o modelo é complexo demais e começa a "decorar" os dados de treino em vez de aprender o padrão geral. O erro de treino é baixíssimo, mas o erro de teste começa a subir, pois o modelo não consegue generalizar para novos dados. É como matar uma mosca com uma bazuca. Nesse ponto, o treinamento deve parar ou o modelo deve ser simplificado.

O cenário ideal é encontrar o ponto ótimo onde o erro de teste é minimizado antes de começar a subir (início do overfitting), equilibrando a capacidade do modelo de aprender os dados de treino e sua capacidade de generalizar para dados novos.

Para ilustrar a prática de treinamento de forma lúdica, recomenda-se o uso da ferramenta "Teachable Machine" do Google, que permite treinar modelos de classificação de imagens de forma visual, reforçando o conceito de apresentar exemplos rotulados para a máquina aprender a distinguir classes.

Assista no

4. Métricas de Avaliação e Matriz de Confusão

Uma vez treinado o modelo, precisamos de métodos objetivos para quantificar sua qualidade. Neste capítulo, focamos na avaliação de modelos de classificação, utilizando a **Matriz de Confusão** e as métricas derivadas dela, com atenção especial para o problema de classes desbalanceadas.

A Matriz de Confusão é uma tabela que cruza os valores reais com os valores preditos pelo modelo. Em um problema binário (ex: Classe 1 vs. Classe 0), temos quatro situações possíveis:

1. **Verdadeiro Positivo (TP):** A classe real era 1 e o modelo previu 1. (Acerto)
2. **Verdadeiro Negativo (TN):** A classe real era 0 e o modelo previu 0. (Acerto)
3. **Falso Positivo (FP):** A classe real era 0, mas o modelo previu 1. (Erro Tipo 1)
4. **Falso Negativo (FN):** A classe real era 1, mas o modelo previu 0. (Erro Tipo 2)

A gravidade desses erros depende do contexto. Em um diagnóstico médico de câncer, um Falso Negativo (dizer que a pessoa não tem câncer quando ela tem) é gravíssimo, pois impede o tratamento. Já um Falso Positivo causaria susto e exames extras, mas não a morte. Portanto, diferentes problemas exigem a otimização de diferentes métricas.

A partir da matriz, calculamos métricas como:

- **Precision (Precisão):** De tudo o que o modelo previu como Classe 1, quanto ele acertou? (TP / TP + FP).
- **Recall (Sensibilidade):** De tudo o que *realmente* era Classe 1 no mundo real, quanto o modelo conseguiu capturar? (TP / TP + FN).

Um ponto crucial discutido é a importância de definir corretamente a **Classe 1 (Positiva)** como a classe de maior interesse para o negócio (ex: o aluno evadido, a doença, a fraude). As métricas padrões geralmente focam na Classe 1.

O capítulo destaca o **Paradoxo da Acurácia** em dados desbalanceados. Imagine uma base onde 900 alunos são persistentes (Classe 0) e apenas 100 são evadidos (Classe 1). Se o modelo simplesmente "chutar" que *todos* os alunos são persistentes (Classe 0), ele terá uma **Acurácia de 90%**. O número parece excelente, mas o modelo é inútil, pois errou 100% dos casos de evasão (Classe 1), que era justamente o que queríamos prever. O modelo sofreu overfitting na classe majoritária.

Nesses casos, a acurácia é uma métrica enganosa. Devemos olhar para métricas que ponderam os erros nas duas classes ou focam na classe minoritária, como o **F1-Score**, o **G-Mean** (média geométrica entre sensibilidade e especificidade) ou métricas balanceadas. O Recall da classe minoritária, por exemplo, revelaria que o modelo está acertando muito pouco (ou nada) do que interessa, desmascarando a falsa sensação de sucesso dada pela acurácia alta. Resolver esse problema pode exigir técnicas de balanceamento de dados ou algoritmos mais robustos ao desbalanceamento.

5. Interpretabilidade, AutoML e Hiperparâmetros

Além de prever com precisão, muitas vezes precisamos entender o *porquê* da previsão. Este capítulo aborda a **Interpretabilidade** (ou explicabilidade) dos modelos, bem como ferramentas modernas de automação (AutoML) e a otimização de hiperparâmetros.

A **Interpretabilidade** visa abrir a "caixa preta" dos modelos. Cada algoritmo oferece formas diferentes de visualização. Na Regressão Logística, podemos analisar os pesos atribuídos a cada variável. Se a "nota de matemática" tem um peso positivo alto associado à classe 0 (persistente), sabemos que quanto maior a nota, maior a chance do aluno ficar. Se a "idade" tem um peso que empurra para a classe 1 (evasão), entendemos que alunos mais velhos têm maior risco de evadir. Nas Árvores de Decisão, podemos visualizar quais atributos foram escolhidos para os "nós" de decisão, indicando sua importância na separação das classes. Existem algoritmos avançados, como SHAP values, dedicados inteiramente a explicar a contribuição de cada atributo para a saída do modelo.

Em seguida, o capítulo introduz o conceito de **AutoML (Automated Machine Learning)**. Ferramentas como **PyCaret**, **H2O** e **AutoGluon** buscam automatizar a pipeline de aprendizagem supervisionada. Com poucas linhas de código, essas bibliotecas realizam pré-processamento, treinam dezenas de modelos diferentes, comparam métricas e selecionam o melhor modelo automaticamente. Elas também podem gerar gráficos de interpretabilidade. Embora sejam ferramentas poderosas que agilizam o trabalho, o cientista de dados deve ter cautela e conhecimento para validar se o que a ferramenta produziu faz sentido.

Por fim, discutimos a **Otimização de Hiperparâmetros**. Como visto anteriormente, hiperparâmetros são configurações de treinamento (ex: profundidade da árvore, tipo de função de custo, número de vizinhos no KNN). O desempenho do modelo depende drasticamente dessas escolhas. O processo de otimização (Tuning) envolve buscar, de forma sistemática, a melhor combinação desses valores para maximizar uma métrica de performance (como AUC ou Acurácia). Os frameworks de AutoML frequentemente já incluem etapas de "fine-tuning" para refinar o modelo vencedor.

Unidade 8.5

Tecnologia em Sistemas para Internet

Assista no

6. Implementação Prática com Scikit-Learn

Neste capítulo, materializamos os conceitos teóricos através da implementação de código utilizando a biblioteca Python **Scikit-Learn**. O foco é demonstrar a pipeline completa, da carga dos dados ao deploy.

O processo prático inicia com a **Importação e Preparação dos Dados**. Selecionamos atributos numéricos e categóricos e definimos a variável alvo (Classe). Um passo importante demonstrado é a inversão dos rótulos para garantir que a classe de interesse (aluno evadido) seja a Classe 1, facilitando a leitura das métricas posteriormente.

Realizamos a **Divisão Treino/Teste** utilizando a função train_test_split. Aqui, utiliza-se o parâmetro stratify, que é essencial para dados desbalanceados, pois garante que a proporção de alunos evadidos e persistentes seja a mesma tanto no conjunto de treino quanto no de teste, mantendo a representatividade estatística.

Para o pré-processamento, adotamos a estrutura de **Pipeline** do Scikit-Learn. Isso permite encadear passos de forma organizada:

1. Para dados numéricos: Imputação de valores ausentes (pela mediana) e escalonamento (MinMax).
2. Para dados categóricos: Imputação (pela moda) e codificação (OneHotEncoding).

A criação do modelo torna-se simples: instanciamos o algoritmo (ex: LogisticRegression ou DecisionTreeClassifier) dentro do pipeline e chamamos o método .fit() com os dados de treino. O modelo aprende "sozinho" nesse passo.

A **Avaliação** é feita chamando o método .predict() nos dados de teste e comparando com as respostas reais. O código demonstra como gerar a matriz de confusão e o relatório de métricas. Observamos na prática o problema do desbalanceamento: a acurácia global foi altíssima (acima de 90%), mas as métricas de Precision e Recall para a classe de evasão foram baixas, confirmando que o modelo, apesar de "acurado" no geral, é ruim para identificar o aluno evadido.

O capítulo também cobre a **Interpretabilidade no Código**, extraindo e plotando os coeficientes da regressão logística (mostrando a influência positiva das notas na persistência) e a importância das features na árvore de decisão.

Finalmente, abordamos a **Persistência e Deploy**. Uma vez treinado, o modelo (que é um objeto Python) pode ser salvo em um arquivo binário (usando pickle ou joblib). Isso permite que o treinamento (que pode demorar horas) seja feito offline. A aplicação final apenas carrega esse arquivo binário e faz predições. Foi demonstrado um exemplo de aplicação web usando **Streamlit**, onde o usuário insere os dados do aluno em um formulário, o sistema carrega o modelo salvo e exibe a probabilidade de evasão on-line.

7. Automação com AutoML (PyCaret e AutoGluon)

O capítulo final apresenta o uso prático de frameworks de **AutoML** para agilizar e robustecer o processo de modelagem, utilizando o **PyCaret** e o **AutoGluon**.

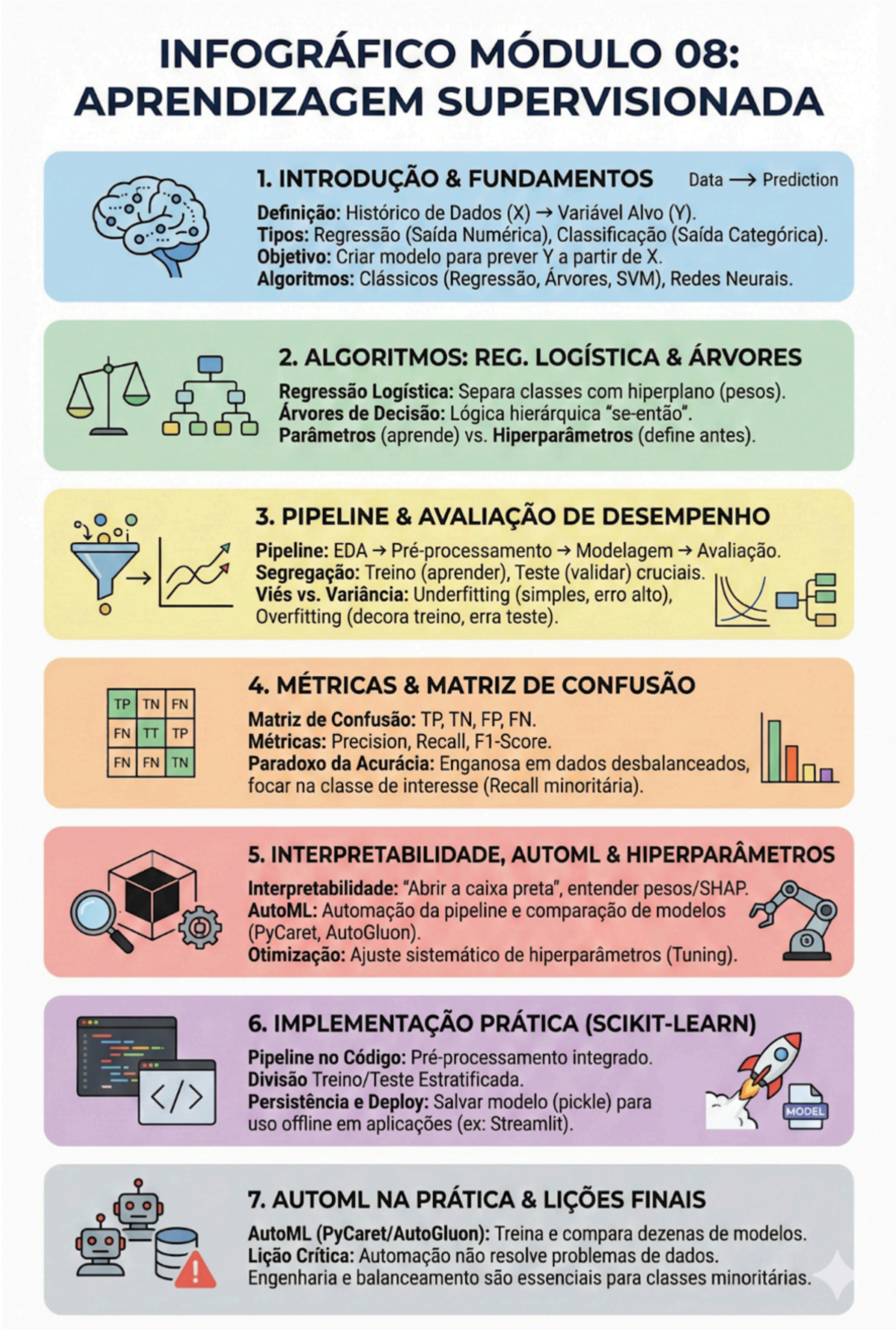
O **PyCaret** é destacado por sua simplicidade e poder. Com ele, a configuração do ambiente (setup) faz automaticamente a inferência dos tipos de dados, limpeza e preparação. O comando compare_models() é o coração da ferramenta: ele treina e avalia simultaneamente uma vasta gama de algoritmos (16 modelos no exemplo, incluindo Random Forest, Gradient Boosting, SVM, etc.) e os ordena por uma métrica de escolha (como AUC ou Acurácia). Isso poupa ao cientista de dados o trabalho manual de testar um por um. Após identificar o melhor modelo (no exemplo, um Gradient Boosting), é possível usar a função tune_model() para otimizar automaticamente os hiperparâmetros.

O **AutoGluon** é apresentado como uma alternativa mais moderna e robusta, suportando versões mais recentes do Python e focando pesadamente em técnicas de **Ensemble** (combinar vários modelos para uma decisão final, seja por votação ou média). O AutoGluon testa dezenas de modelos e empilha os melhores resultados para maximizar a performance.

No entanto, a lição crítica deste capítulo final é que **a automação não faz milagres**. Mesmo testando dezenas de modelos de ponta com AutoML, os resultados para a classe minoritária (evasão) continuaram insatisfatórios no exemplo prático. Isso reforça que o problema não era a falta de um algoritmo poderoso, mas sim questões relacionadas aos **dados**: desbalanceamento severo e talvez a falta de atributos explicativos suficientes (poucas colunas).

A conclusão é que o AutoML é uma ferramenta excelente para produtividade e benchmarking, mas o conhecimento do especialista em manipular os dados (Data Engineering), balanceá-los e criar novas features é insubstituível para resolver problemas complexos de viés e baixa performance em classes específicas.

Unidade 8.7
Tecnologia em Sistemas para Internet

Assista no

8. Infográfico da Unidade 8